

Sky Analytics Stream

Architettura Distribuita per l'Analisi in Tempo Reale del Traffico Aereo

Flavio Simonelli

Matricola 0365333

Dipartimento di Ingegneria

Università di Roma "Tor Vergata"

flavio.simonelli@alumni.uniroma2.eu

Francesco Masci

Matricola 0365258

Dipartimento di Ingegneria

Università di Roma "Tor Vergata"

francesco.masci.02@alumni.uniroma2.eu

Sommario—Questo lavoro presenta *Sky Analytics Stream*, un'architettura distribuita ed elastica per l'analisi e il monitoraggio in tempo reale del traffico aereo statunitense basata su Apache Flink e Apache Kafka. Il sistema riceve i dati da un simulatore ad alte prestazioni sviluppato in Go, in grado di riprodurre flussi di eventi storici in event-time con disordine controllato e fattori di accelerazione variabili. Attraverso un'unica topologia a grafo diretto aciclico (DAG), la pipeline esegue in parallelo tre analisi: aggregazioni orarie delle performance delle compagnie aeree, classifiche in tempo reale (Top-10) degli aeroporti con ritardi severi e stima dei percentili dei ritardi tramite l'algoritmo DDSketch. Risultati e metriche di telemetria sono storicizzati in InfluxDB e visualizzati tramite Grafana. La tolleranza ai guasti è garantita da checkpointing *exactly-once* con stato incrementale gestito da RocksDB. Il sistema, testato sia localmente che su cluster Amazon EC2, sostiene throughput superiori a 9.000 eventi/secondo. La valutazione sperimentale analizza le prestazioni del sistema al variare del parallelismo, del disordine degli eventi e della frequenza dei checkpoint.

Index Terms—Apache Flink, Stream Processing, Apache Kafka, Traffico Aereo, Percentili Approssimati, Tolleranza ai Guasti, DDSketch, checkpointing.

I. INTRODUZIONE

L'analisi tempestiva del traffico aereo richiede di elaborare flussi continui di dati per aggiornare metriche prestazionali e classifiche di ritardo in tempo reale. In questo contesto, l'accuratezza e la freschezza dei risultati dipendono dalla capacità di gestire le sfide intrinseche dello stream processing. In particolare, ritardi di rete, colli di bottiglia nell'ingestione e arrivi fuori ordine fanno sì che il tempo logico in cui un volo si verifica (*event-time*) non venga rispettato nell'ordine di come gli eventi arrivano ai processori di elaborazione. Per fornire risultati coerenti e completi, è dunque cruciale definire politiche di watermarking e di gestione dei dati tardivi che bilancino correttamente la latenza e l'accuratezza delle aggregazioni.

Per affrontare tali sfide, questo lavoro presenta il progetto e la valutazione sperimentale di *Sky Analytics Stream*, un'architettura distribuita progettata per elaborare un dataset storico di circa 2,2 milioni di voli negli Stati Uniti (relativi al periodo gennaio–aprile 2025). Il dataset viene riprodotto in modo controllato tramite un simulatore ad alte prestazioni sviluppato in Go, in grado di iniettare dati in Apache Kafka applicando fattori di compressione temporale e introducendo disordine artificiale. La pipeline di elaborazione principale è

realizzata in Apache Flink e implementa un singolo grafo aciclico (DAG) ottimizzato per eseguire in parallelo tre analisi distinte:

- 1) Il calcolo orario delle statistiche di volo per ciascuna compagnia aerea;
- 2) Il tracciamento dinamico delle classifiche (Top-10) degli aeroporti con i maggiori ritardi accumulati;
- 3) La stima continua della distribuzione dei ritardi complessivi.

Il resto della relazione è organizzato come segue: la Sezione ?? descrive l'architettura generale del sistema e il flusso dei dati; la Sezione ?? approfondisce i dettagli implementativi delle singole componenti; la Sezione ?? illustra l'infrastruttura di deployment; la Sezione ?? presenta la valutazione sperimentale e i risultati dei test prestazionali; infine, la Sezione ?? conclude il report delineando possibili sviluppi futuri.

II. ARCHITETTURA DEL SISTEMA

L'architettura di *Sky Analytics Stream* è concepita come una pipeline multi-tier per lo stream processing distribuito, orientata ad alta scalabilità, robustezza e tolleranza ai guasti. Il sistema adotta un approccio a microservizi disaccoppiati in cui ciascun componente presiede a una specifica fase della pipeline: generazione, brokeraggio, elaborazione, persistenza e visualizzazione, come illustrato nello schema logico di Fig. ??.

A. Lo Stack Tecnologico (Big Data Stack)

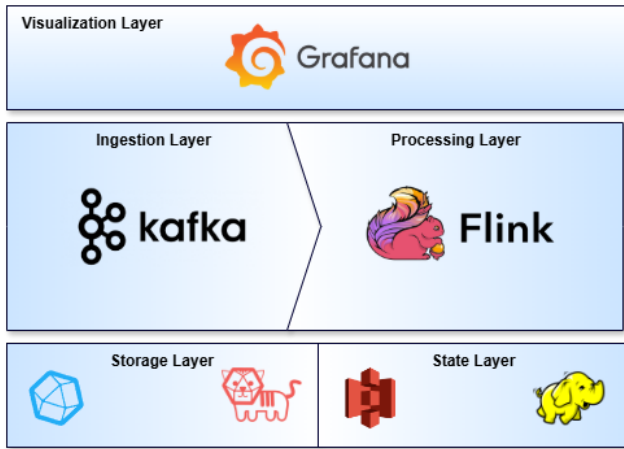


Figura 1. Flusso dei dati e integrazione dei framework in Sky Analytics Stream.

La pipeline si articola in sei livelli architetturali principali:

- 1) **Ingestion Layer (Go Simulator):** Un modulo sviluppato in Go che simula la sorgente in tempo reale partendo dal dataset storico locale. Ottimizzato per alte prestazioni, funge da produttore Kafka.
- 2) **Message Broker Layer (Apache Kafka):** Il bus dei messaggi distribuito che funge da buffer. Disaccoppia la sorgente di dati dal motore di elaborazione e quest'ultimo dal consumatore Telegraf della suite TICK, attenuando i picchi di carico e gestendo in modo trasparente la contropressione (*backpressure*).
- 3) **Stream Processing Layer (Apache Flink):** Il nucleo computazionale del sistema. Esegue una topologia distribuita consumando i dati da Kafka ed eseguendo elaborazioni stateful in event-time.
- 4) **State Storage Layer (HDFS / RocksDB):** Lo storage distribuito deputato al salvataggio dei checkpoint. RocksDB funge da backend di stato locale (in-memory/su disco locale del worker), mentre HDFS garantisce la persistenza distribuita dei backup di stato per la tolleranza ai guasti.
- 5) **Data Integration & Storage Layer (Telegraf & InfluxDB):** InfluxDB memorizza sia i risultati analitici che le metriche di runtime di Flink. Telegraf funge da connettore reattivo leggendo i risultati da Kafka e inserendoli in InfluxDB.
- 6) **Visualization Layer (Grafana):** Fornisce l'interfaccia utente finale per l'osservabilità delle performance del cluster e il monitoraggio degli indicatori analitici del traffico aereo.

La Tabella ?? riassume i componenti tecnologici impiegati e i rispettivi ruoli nel sistema.

B. Flusso dei Dati (Data Flow)

Il ciclo di vita di un singolo evento di volo attraversa sequenzialmente l'intera pipeline seguendo il flusso descritto

di seguito:

- 1) **Generazione e Replay:** Il simulatore Go legge i dati grezzi in streaming direttamente dal dataset compresso. Gli eventi vengono ordinati per data/ora logica di volo, convertiti in JSON e inviati al topic Kafka `flights-stream` con una frequenza che rispetta lo *Speedup Factor* impostato. Qualora configurato, viene applicato l'algoritmo di sfasamento temporale per inserire disordine nello stream.
- 2) **Brokeraggio:** Kafka distribuisce i messaggi in ingresso sulle partizioni del topic `flights-stream`. La scelta di un partizionamento multiplo consente a Flink di leggere i dati in parallelo.
- 3) **Elaborazione:** I TaskManager di Apache Flink consumano lo stream di eventi. Il motore gestisce i watermark in base all'event-time inserito nei record e calcola i risultati per le tre query analitiche concorrenti. Man mano che le finestre temporali si chiudono, Flink emette i risultati intermedi e finali, scrivendoli in specifici topic Kafka di output (es. `flights-q1-results`, `flights-q2-1h-results`, ecc.).
- 4) **Fault-Tolerance:** Durante l'elaborazione, Flink esegue periodicamente checkpoint asincroni. Lo stato locale delle finestre e degli sketch (memorizzato in RocksDB) viene sincronizzato su HDFS. In caso di fallimento di un TaskManager, il JobManager ripristina lo stato dall'ultimo checkpoint valido su HDFS, riavviando il consumo da Kafka dall'offset corretto.
- 5) **Raccolta e Persistenza:** Telegraf ascolta i topic di output di Kafka e converte i messaggi JSON in record pronti per InfluxDB, inserendoli nel bucket dei risultati. Parallelamente, il reporter InfluxDB integrato in Flink invia ogni secondo le metriche di telemetria ad un bucket separato in InfluxDB.
- 6) **Visualizzazione:** Grafana interroga costantemente InfluxDB tramite il linguaggio Flux/InfluxQL, aggiornando in tempo reale le visualizzazioni grafiche delle dashboard destinate sia alle metriche applicative (es. ritardi delle compagnie) sia a quelle infrastrutturali (es. latenza end-to-end e carico del cluster).

C. Scelte Architetturali Chiave

La scelta di questa specifica configurazione architetturale risponde a precisi requisiti ingegneristici:

- **Disaccoppiamento tramite Code di Messaggi:** L'uso di Kafka come intermediario protegge Flink da variazioni repentine del throughput di rete o della sorgente. Kafka funge da buffer persistente, consentendo a Flink di consumare i dati alla propria velocità massima stabile ed esercitando contropressione nativa se necessario.
- **Condivisione della Sorgente:** Anziché lanciare tre job Flink distinti (ognuno dei quali dovrebbe consumare lo stream da Kafka separatamente, decodificare i messaggi JSON ed emettere watermark), le tre query analitiche sono modellate all'interno di un unico DAG. In questo modo si dimezza il carico di I/O su Kafka e si condivide

Tabella I
LO STACK TECNOLOGICO DI SKY ANALYTICS STREAM

Layer	Componente	Tecnologia	Ruolo e Scopo Principale
Ingestion	Simulatore di Flusso	Go Application	Lettura zero-copy, ordinamento, serializzazione binaria e riproduzione accelerata con disordine controllato.
Message Broker	Event Bus	Apache Kafka	Disaccoppiamento produttore-consumatore, partizionamento del flusso (<i>flights-stream</i>) e storage temporaneo dei risultati.
Processing	Stream Engine	Apache Flink	Consumo parallelo multi-partizione, calcolo di aggregati in event-time e scrittura su Kafka.
State Storage	Distributed File System	Apache HDFS	Storage durevole e centralizzato per il salvataggio dei checkpoint e savepoint di Flink.
Data Integration Persistence	Metrics & Results Collector Time-Series DB	Telegraf Agent InfluxDB v2	Ingestione automatica dei risultati analitici da Kafka verso InfluxDB. Storicizzazione efficiente dei risultati delle query e telemetria del cluster Flink (CPU, throughput, latenza).
Visualization	Dashboarding UI	Grafana	Visualizzazione real-time delle metriche di traffico aereo e monitoraggio dello stato di salute dell'infrastruttura.

la fase iniziale di parsing ed estrazione del timestamp logico, ottimizzando l'uso di memoria e CPU sui worker.

- **Archiviazione Stato Ibrida (RocksDB + HDFS):** La scelta di RocksDB come State Backend consente di gestire uno stato applicativo di dimensioni superiori alla memoria RAM disponibile dei TaskManager, riversando i dati su disco locale. L'integrazione con HDFS garantisce che tali dati siano protetti contro la perdita del singolo host tramite la replica distribuita dei file di checkpoint.
- **Canale di Output Asincrono per l'Osservabilità:** Flink scrive i risultati su Kafka anziché effettuare chiamate dirette a un database. Questo evita di bloccare i thread di calcolo di Flink con la latenza di scrittura sincrona di un DB, delegando l'inserimento batch a Telegraf e preservando le performance computazionali.

III. IMPLEMENTAZIONE DELLE COMPONENTI

A. Simulatore di Ingestione in Go

L'ingestione e il replay degli eventi storici in tempo reale sono gestiti da un componente custom sviluppato in linguaggio **Go**. La scelta tecnologica è motivata dalla necessità di disporre di un agente di ingestione ad altissime prestazioni, caratterizzato da un consumo minimo di risorse e in grado di garantire throughput elevati senza introdurre colli di bottiglia o latenze spurie dovute alla Garbage Collection (frequenti in ecosistemi basati su JVM).

Il simulatore implementa diverse funzionalità avanzate e ottimizzazioni ingegneristiche:

- **Ingestione Streaming Zero-Copy ed Integrity Check:** Al primo avvio, il simulatore tenta di localizzare l'archivio del dataset (in formato *.tar.gz*) in una directory locale configurata tramite la variabile d'ambiente `INPUT_ARCHIVE_PATH`. Qualora non presente, l'applicazione lo scarica automaticamente tramite richiesta HTTP dal server del dipartimento¹. Successivamente, viene verificata l'integrità del file confrontandone l'hash SHA-1 con quello atteso (17be276b72bd987e

72598b0ea4907c3b19350606). Una volta superato il controllo, l'archivio originale viene decodificato al volo in memoria tramite i pacchetti nativi di Go (*compress/gzip* e *archive/tar*), senza la necessità di estrarre preventivamente su disco circa 2.2 milioni di righe in formato CSV. Questo flusso zero-copy riduce al minimo l'uso dello spazio di archiviazione ed elimina i tempi morti dovuti alle operazioni di I/O su disco.

- **Binary Caching in formato GOB:** Al fine di minimizzare i tempi di inizializzazione nelle successive esecuzioni, gli eventi letti dal file CSV vengono convertiti in una slice di strutture tipizzate Go di tipo `models.FlightRecord`. I record vengono quindi ordinati cronologicamente rispetto al loro event-time logico e serializzati in un file di cache binario locale (*.gob*) utilizzando il codificatore nativo `encoding/gob` di Go.
- **Gestione del Tempo Fisico e Replay ad Alta Precisione (Sleep-Spinlock):** Per ciascun volo, l'event-time logico viene ricostruito combinando i campi `YEAR`, `MONTH`, `DAY_OF_MONTH` e l'orario programmato `CRS_DEP_TIME`. Il simulatore emula la distanza temporale reale tra i voli applicando un fattore di compressione del tempo (*Speedup Factor*) ed attendendo la differenza relativa (Δt) riscalata rispetto all'evento precedente. Con alti fattori di accelerazione (dove le finestre temporali logiche di un'ora si contraggono in pochi millisecondi reali), le funzioni di *sleep* del sistema operativo risulterebbero inadeguate a causa dell'overhead dello scheduler del kernel e dei *context switch*. Per ovviare a ciò, è stata implementata una strategia di attesa ibrida (*Sleep-Spinlock*): se l'attesa supera la soglia critica (*spin threshold* nel file di configurazione), il thread esegue una sleep cooperativa rilasciando la CPU, mentre al di sotto di tale valore effettua un **spinlock** (attesa attiva). Questo impedisce la sospensione del processo da parte del kernel nel momento critico, garantendo precisioni nell'ordine dei microsecondi ed assicurando l'uniformità del throughput programmato.
- **Generazione di Disordine (Out-of-Order Injection):** Al fine di stressare e validare le strategie di Watermarking e

¹<http://www.ce.uniroma2.it/courses/sabd2526/project/project-1-data.tar.gz>

la gestione dei dati tardivi (*allowed lateness*) di Apache Flink e analizzare il sistema in scenari realistici dove le sorgenti sono diversificate e poco affidabili, il simulatore implementa un decoratore basato su un algoritmo probabilistico. Con una probabilità definita $P \in [0, 1]$, un record viene selezionato e il suo istante di pubblicazione viene dislocato in avanti nel tempo di un offset casuale limitato da un valore massimo configurabile (δ_{max} in minuti logici). La sequenza di eventi risultante viene infine ri-ordinata sul tempo di pubblicazione reale prima dell'invio, riproducendo accuratamente i tipici disallineamenti di rete di uno stream distribuito.

- **Pubblicazione asincrona su Kafka:** i record sono serializzati in JSON mantenendo i nomi delle colonne BTS e inviati al topic `flights-stream` mediante `kafka-go`. Il writer usa bilanciamento round-robin, batch fino a 100 messaggi o 100 kB, timeout di 2 ms e modalità asincrona, evitando che la conferma di ogni singolo messaggio alteri la temporizzazione del replay. In alternativa, un sink testuale consente test locali senza Kafka per sviluppo e debugging.

B. Infrastruttura di Disaccoppiamento ed Event Bus: Apache Kafka

Per disaccoppiare la sorgente dei dati (il simulatore Go) dal motore di stream processing (Apache Flink), è stato integrato **Apache Kafka** come message broker distribuito ad alto throughput. Kafka funge da polmone architetturale, assorbendo le fluttuazioni di carico del flusso in ingresso ed esponendo i dati in modo ordinato e persistente.

1) Organizzazione dei Topic e Partizionamento:

Il flusso di dati in ingresso è gestito tramite il topic principale `flights-stream`, configurato con 6 partizioni parallele. Il partizionamento consente di parallelizzare l'elaborazione su più slot di esecuzione di Flink, garantendo la scalabilità orizzontale del sistema. Per la storizzazione dei risultati analitici prodotti dalle tre query concorrenti, sono stati definiti diversi topic di output dedicati per ciascuna finestra e granularità temporale (es. `flights-q1-results`, `flights-q2-1h-results`, `flights-q3-1d-results`, ecc.). Questo disaccoppia ulteriormente Flink dai database di storage a valle, permettendo a componenti ausiliari come Telegraf di consumare i risultati in modalità asincrona.

2) *Fase di Ingestione in Flink (Kafka Source):* L'integrazione di Kafka come sorgente dati in Flink avviene tramite la classe nativa `KafkaSource`, istanziata mediante un builder personalizzato (`SourceBuilder`). Il consumo dello stream parte dall'offset meno recente (`OffsetsInitializer.earliest()`).

Per mappare i byte grezzi transitati su Kafka in oggetti Java tipizzati, è stato implementato uno schema di deserializzazione custom, `FlightRecordDeserializationSchema` (che implementa l'interfaccia `KafkaRecordDeserializationSchema<FlightRecord>`).

Tale classe esegue le seguenti operazioni per ogni record consumato:

- **Deserializzazione JSON:** Utilizza una libreria Jackson `ObjectMapper` (dichiarata `transient` e istanziata `lazy` per evitare errori di serializzazione nei `TaskManager`) per decodificare il payload del messaggio in un oggetto `FlightRecord`, scartando in modo tollerante eventuali record corrotti o messaggi vuoti (tombstone);
- **Estrazione dell'Event-Time:** Calcola l'epoch millisecondi logico associato al volo (`eventTimeMillis`) combinando i campi `data` e `ora` del record;
- **Iniezione del Tempo di Ingestione:** Inietta nel record il timestamp fisico corrente del sistema (`systemIngestionTime`) tramite `System.currentTimeMillis()`. Questo passaggio è cruciale per misurare accuratamente la latenza end-to-end della pipeline a valle.

3) *Fase di Scrittura dei Risultati (Kafka Sink):* Le tre query analitiche concorrenti producono flussi di risultati che vengono re-iniettati in Kafka tramite istanze di `KafkaSink`, configurate con la garanzia di consegna `DeliveryGuarantee.AT_LEAST_ONCE` per garantire la tolleranza ai guasti senza l'overhead delle transazioni a due fasi.

La serializzazione da oggetti Java a payload JSON pronti per essere trasmessi in rete è delegata a schemi di serializzazione personalizzati (come `AirlinePerformanceRecordSerializer` per la Query 1 e `DelayDistributionRecordSerializer` per la Query 3). Ognuno di essi serializza lo stato dell'oggetto in array di byte JSON. Nel caso della Query 1, il codice della compagnia aerea (`OP_UNIQUE_CARRIER`) viene utilizzato esplicitamente come chiave di partizionamento del record Kafka. Questo assicura che gli eventi relativi allo stesso vettore vengano inoltrati alla medesima partizione di Kafka, agevolando eventuali letture downstream ordinate o partizionate per chiave.

C. Motore di Stream Processing: Apache Flink

Il core computazionale dell'applicazione è implementato in **Apache Flink** e strutturato in un unico job distribuito. La topologia a grafo aciclico (DAG) è definita all'interno della classe principale `FlightAnalysisJob` ed è ottimizzata per eseguire in parallelo le tre query analitiche condividendo la sorgente dei dati e la fase di preprocessing, riducendo al minimo l'overhead di I/O e di rete.

1) *Configurazione dell'Environment e Watermarking:* Il job rileva dinamicamente l'ambiente di esecuzione controllando la variabile d'ambiente `FLINK_CONF_DIR`: in caso di esecuzione su cluster distribuito, si collega all'environment corrente; in caso di debug locale, avvia un mini-cluster in-process o invia il pacchetto `fat-jar` a un `JobManager` remoto.

Per la gestione dell'avanzamento del tempo logico, viene configurata una strategia di watermarking basata su event-time (`WatermarkStrategy`) che estrae il timestamp in millisecondi generato dal simulatore. La tolle-

ranza al disordine temporale dei record (introdotti probabilisticamente dal simulatore) è gestita tramite la politica `forBoundedOutOfOrderness`, configurabile in minuti logici. Per evitare blocchi di esecuzione nel caso in cui una delle 6 partizioni Kafka risulti temporaneamente inattiva, la strategia è irrobustita con un timeout di idleness (`withIdleness`), che esclude temporaneamente la partizione dal calcolo del watermark globale.

2) *Meccanismo di Checkpointing e Fault Tolerance*: La tolleranza ai guasti con semantica di elaborazione *exactly-once* è interamente delegata al meccanismo di checkpointing distribuito di Flink. Le opzioni configurate a livello di engine includono:

- **Minimo intervallo di pausa** (`MIN_PAUSE_BETWEEN_CHECKPOINTS`): Previene la saturazione delle risorse di storage ed elaborazione impedendo a Flink di innescare un nuovo checkpoint se il precedente non è terminato da un intervallo minimo specificato (evitando il fenomeno della *checkpoint backpressure*);
- **Unaligned Checkpoints**: Se abilitati da configurazione, consentono di eseguire checkpoint asincroni superando i messaggi bloccati nei buffer dei canali di rete, riducendo i tempi di salvataggio in presenza di forte contropressione;
- **Storage Backend ibrido**: Lo stato locale delle finestre e degli accumulatori è ospitato in-memory o su disco locale tramite **RocksDB** (State Backend). RocksDB supporta salvataggi incrementali asincroni che vengono inviati e replicati su **HDFS** (o Amazon S3 in ambienti cloud) all'URI configurato.

3) *Data Quality e Preprocessing*: Prima di biforcare il flusso di eventi verso le tre query, il flusso grezzo attraversa la classe `PipelinePreprocessing`. Questa esegue un filtro logico-strutturale (`LogicalInconsistencyFilter`) implementato come `RichFilterFunction`:

- Scarta record corrotti, con riferimenti nulli o campi obbligatori mancanti (es. identificativi aeroporto, codice compagnia, orari programmati);
- Rileva e scarta incoerenze semantiche nel dataset (ad esempio, voli contrassegnati contemporaneamente come cancellati e dirottati);
- Raccoglie metriche di telemetria incrementando un contatore Flink personalizzato (`corrupted_records_total`) per il monitoraggio della qualità dei dati e aggiorna un tracciante di latenza basato sulla differenza tra il tempo fisico di elaborazione e il `systemIngestionTime` registrato in fase di decodifica Kafka.

4) *Query 1: Statistiche delle Compagnie Aeree*: La prima query calcola su base oraria le performance dei quattro vettori target (AA, DL, UA, WN). Il flusso viene filtrato tramite `TargetAirlineFilter`, proiettato su oggetti leggeri `AirlinePerformanceEvent` per minimizzare lo stato occupato, e quindi partizionato tramite `keyBy` sul codice della compagnia.

La finestra temporale è di tipo tumbling basata su event-time con durata fissa di 1 ora. Per gestire i record in ritardo senza perdere accuratezza, viene applicata una tolleranza di latenza (`allowedLateness`). I record che giungono oltre questa soglia vengono convogliati in un canale di side-output (`q1-late-flights`) e analizzati da un `LateRecordMetricAnalyzer` per fini statistici. L'aggregazione è implementata combinando:

- 1) **AirlinePerformanceAggregator**: Una `AggregateFunction` che calcola in modo incrementale l'accumulatore dello stato (`AirlinePerformanceAccumulator`) mantenendo in memoria un footprint costante di complessità $O(1)$ per chiave (conteggio voli, cancellazioni, deviazioni, partenze ritardate oltre i 15 minuti e somma dei ritardi di partenza);
- 2) **AirlinePerformanceWindowProcessor**: Una `ProcessWindowFunction` che viene invocata solo alla chiusura della finestra logica. Riceve l'accumulatore pre-calcolato e vi associa i metadati della finestra (inizio e fine) per generare il record finale `AirlinePerformanceResult` da inviare a Kafka.

5) *Query 2: Classifica Top-10 degli Aeroporti con Ritardi Severi*: La seconda query individua i 10 aeroporti di partenza con il maggior numero di ritardi severi (ritardo di partenza > 15 minuti). Il calcolo è effettuato concorrentemente su tre ambiti temporali: finestre tumbling di 1 ora, finestre tumbling di 6 ore e una finestra globale ad accumulo continuo (`GlobalWindows`) che emette risultati parziali a intervalli regolari (es. ogni 24 ore logiche) tramite un trigger temporale continuo (`ContinuousEventTimeTrigger`).

La topologia adotta una strategia di **classificazione** in due fasi logiche per ottimizzare il throughput e gestire il disordine:

- **Fase 1: Aggregazione Locale**: Lo stream è partizionato per `originAirportId`. Su ciascuna partizione viene aperta la finestra temporale e calcolato in modo incrementale il numero di partenze ritardate dell'aeroporto tramite `RankAirportsAggregator`. Alla chiusura della finestra, viene emesso un record parziale contenente il totale dei ritardi e la media dei ritardi per quell'aeroporto.
- **Fase 2: Ranking Globale e Timer**: I risultati della prima fase sono ri-partizionati per tempo di fine della finestra (`windowEnd`), convergendo su istanze parallele della classe `RankAirportsRankProcessor` (una `KeyedProcessFunction`). Per evitare di calcolare e pubblicare classifiche parziali prima di aver ricevuto tutti i dati tardivi, il processore memorizza i record parziali in un `MapState` strutturato per aeroporto e **registra** un timer di event-time impostato a:

$$t_{\text{timer}} = \text{windowEnd} + \text{allowedLateness} + 1 \text{ secondo} \quad (1)$$

- **Invocazione del Timer e Heap Sort**: Al trigger del timer (quando Flink garantisce che nessun altro evento appartenente a quella finestra possa essere processato),

viene invocato il metodo `onTimer`. Il processore estrae gli aeroporti dal `MapState`, filtra quelli che hanno registrato meno di 30 voli totali per garantire la rilevanza statistica del campione, e inserisce i restanti in una ****coda con priorità**** (Min-Heap) di dimensione fissa pari a 10.

I criteri di ordinamento nel Min-Heap, comprensivi di tie-breaker deterministici, sono i seguenti:

- 1) Numero di voli con ritardo severo (decrescente);
- 2) In caso di parità, ritardo medio di partenza (decrescente);
- 3) In caso di ulteriore parità, ID numerico dell'aeroporto (crescente).

Al termine, la coda viene svuotata per popolare la lista ordinata finale, a ciascun aeroporto viene assegnato il rank da 1 a 10, lo stato locale viene ripulito per prevenire memory leak e i record ordinati vengono scritti sul topic Kafka di competenza.

6) *Query 3: Stima della Distribuzione dei Ritardi (DD-Sketch)*: La terza query stima la distribuzione dei ritardi (percentili p25, p50, p75, p90, p99) per ciascuna compagnia aerea in base alla fascia oraria di partenza programmata (suddivisa in 24 slot da 0 a 23). L'elaborazione avviene su finestre tumbling di 1 giorno, 7 giorni e su una finestra globale cumulativa. Lo stream è partizionato per una chiave composta `Tuple2<OP_UNIQUE_CARRIER, DepartureHour>`.

A causa dell'alto volume di eventi e della cardinalità delle chiavi, mantenere in memoria l'elenco di tutti i ritardi per calcolare i percentili esatti sarebbe impraticabile. Per risolvere questo problema, è stata integrata la struttura dati probabilistica ****DDSketch**** tramite la classe `DelayDistributionAccumulator`:

- **Accuratezza Relativa Bounded:** La mappatura adotta un'interpolazione cubica (`CubicallyInterpolatedMapping`) con accuratezza relativa impostata a $\alpha = 0.01$ (che garantisce un errore massimo stimato entro l'1% del valore reale del percentile);
- **Store memory-bounded:** Per prevenire una crescita incontrollata dello stato locale in RocksDB indotta da ritardi anomali (outlier estremi), lo sketch limita i bin di memorizzazione tramite la classe `CollapsingLowestDenseStore` impostata a un massimo di 2048 bin. I bin con valori inferiori vengono collassati per primi, preservando l'accuratezza sui ritardi più elevati (code della distribuzione);
- **Serialization personalizzata:** Poiché la classe `DDSketch` di Datalog non è nativamente ottimizzata per la serializzazione Java distribuita, l'accumulatore ne esclude i campi marcandoli come `transient` e implementa routine di serializzazione binaria custom mediante l'override dei metodi standard JVM `writeObject` e `readObject`. Questo riduce drasticamente lo spazio occupato dai checkpoint su HDFS e i tempi di trasferimento di rete.

IV. INFRASTRUTTURA DI DEPLOYMENT

La pipeline di *Sky Analytics Stream* è stata progettata per supportare due modalità di esecuzione speculari ma dimensionate diversamente: un deployment locale a container singolo/multiplo per sviluppo, e un deployment distribuito multi-nodo su infrastruttura cloud per test prestazionali ed esperimenti di scalabilità.

A. Deployment Locale (Docker Compose)

La configurazione locale consente di validare la correttezza logica del codice e testare il comportamento del sistema in scenari controllati. Viene realizzata tramite un file `docker-compose.yml` che orchestra undici container connessi a una rete privata bridge (`sae-net`):

- **Ingestion Broker:** Un container per il broker Kafka (versione 4.3.1) e un container di inizializzazione (`init-kafka`) che crea i topic necessari impostandovi il rispettivo partizionamento (6 partizioni per lo stream di ingresso);
- **Stream Processing:** Un container Flink JobManager e tre container TaskManager. Ogni TaskManager dispone di due slot di elaborazione, per un parallelismo massimo locale di 6;
- **State Storage:** Tre container HDFS (un NameNode e due DataNodes) per supportare il salvataggio dei checkpoint in modalità distribuita locale;
- **Persistence Telemetry:** Un container InfluxDB (versione 2.7) per storicizzare risultati e telemetrie, accoppiato a un container Telegraf che converte i messaggi JSON consumati da Kafka nel modello dati di InfluxDB;
- **Visualization:** Un container Grafana accoppiato a un container `grafana-image-renderer` (usato per esportare e catturare i grafici delle dashboard).

I parametri di configurazione (porte fisiche, credenziali di accesso ai database, token di telemetria e parametri di accelerazione) sono iniettati tramite variabili d'ambiente caricate da un file locale `.env`.

B. Deployment Distribuito su AWS EC2

Per condurre le campagne sperimentali in condizioni di carico reale e misurare la reale scalabilità orizzontale, il sistema è distribuito su un cluster multi-nodo in ambiente cloud ****Amazon Web Services (AWS)****.

1) *Provisioning dell'Infrastruttura (CloudFormation Route 53)*: Il deployment su AWS è automatizzato tramite script Bash (`deploy-network.sh` e `deploy-node.sh`) che richiamano template di ****AWS CloudFormation****. L'orchestratore esegue le seguenti fasi:

- 1) **Network Stack:** Crea una VPC dedicata, una sottorete pubblica con CIDR associato, una Security Group con regole di ingresso/uscita per le porte dei servizi, e una ****Private Hosted Zone**** su ****Amazon Route 53**** con dominio privato `flight-analysis.local`;
- 2) **Bootstrap ed S3:** I pacchetti pre-compilati (`fat-jar` di Flink), i file di configurazione (`telegraf.conf`) e

le variabili d'ambiente (`.env`) vengono caricati su un bucket `**Amazon S3**` privato;

- 3) **Node Provisioning:** Lancia le istanze EC2. Ciascun nodo esegue uno script di bootstrap (`User Data`) che installa Docker, scarica le configurazioni e i pacchetti dal bucket S3 (utilizzando un ruolo IAM temporaneo assegnato all'istanza) e avvia i container Docker locali. I nodi si registrano automaticamente su Route 53 per consentire la risoluzione dei nomi privati (es. `jobmanager.flight-analysis.local`).

2) *Dimensionamento delle Istanze EC2 (Macchine utilizzate):* Il cluster distribuito è composto da istanze EC2 appartenenti alla famiglia general-purpose (`t3`) e compute-optimized (`c6i`) di AWS, dimensionate per bilanciare i costi con la necessità di risorse CPU e memoria:

- **Kafka Cluster** (3 Istanze):

- *Broker 1 (Voter/Controller):* Istanza `**t3.large**` (2 vCPU, 8 GB RAM) con disco SSD EBS per gestire il carico di scrittura primario e coordinare il quorum;
- *Broker 2-3:* Istanze `**t3.medium**` (2 vCPU, 4 GB RAM) che agiscono come repliche e broker ausiliari.

- **Flink JobManager** (1 Istanza): Istanza `**c6i.large**` (2 vCPU, 4 GB RAM) ottimizzata per il calcolo, adibita al coordinamento dei checkpoint, schedulazione dei task e gestione dell'interfaccia web di monitoraggio;
- **Flink TaskManagers** (3 o più Istanze): Istanze `**t3.medium**` (2 vCPU, 4 GB RAM) adibite all'esecuzione fisica dei task di Flink (ogni TaskManager ospita 2 slot di calcolo);
- **InfluxDB Telegraf Node** (1 Istanza): Istanza `**c6i.large**` (2 vCPU, 4 GB RAM) compute-optimized per far fronte alle intense scritture batch di Telegraf su InfluxDB e interrogazioni da Grafana;
- **Grafana Metrics Node** (1 Istanza): Istanza `**t3.small**` (2 vCPU, 2 GB RAM) per ospitare il servizio di dashboarding;
- **Ingestion/Simulator Node** (1 Istanza): Istanza `**t3.medium**` (2 vCPU, 4 GB RAM) su cui gira il simulatore in Go che invia gli eventi in streaming verso il cluster Kafka.

La risoluzione dei nomi DNS privati (ad esempio `kafka.flight-analysis.local` che mappa tramite record CNAME al broker primario, e `flink.flight-analysis.local` al JobManager) consente alle macchine di dialogare in modo trasparente senza dover hardcodare gli indirizzi IP pubblici o privati variabili delle istanze.

V. VALUTAZIONE SPERIMENTALE

VI. CONCLUSIONI E SVILUPPI FUTURI

RIFERIMENTI BIBLIOGRAFICI

- [1] L. Fernando, H. Bindra, and K. Daudjee, "An experimental analysis of quantile sketches over data streams," in *Proceedings of the 26th International Conference on Extending Database Technology (EDBT '23)*, pp. 1–12, 2023.
- [2] C. Masson, J. E. Rim, and H. K. Lee, "DDSketch: A fast and fully-mergeable quantile sketch with relative-error guarantees," *Proceedings of the VLDB Endowment*, vol. 12, no. 12, pp. 2195–2205, 2019.

APPENDICE A
UTILIZZO DI STRUMENTI DI INTELLIGENZA ARTIFICIALE GENERATIVA

APPENDICE B
RAPPRESENTAZIONI GRAFICHE E ALLEGATI